



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 18

1. ENUNCIADO

Reimplemente a tarefa 17, agora distribuindo as colunas entre os processos. Utilize apenas em uma versão `MPI_Type_vector` e em outra acrescente `MPI_Type_create_resized` para definir um tipo derivado que represente colunas da matriz. Use `MPI_Scatter` com esse tipo para distribuir blocos de colunas, e `MPI_Scatter` para enviar os segmentos correspondentes de `x`.

Cada processo deve calcular uma contribuição parcial para todos os elementos de `y` e usa `MPI_Reduce` com `MPI_SUM` para somar os vetores parciais no processo 0.

Discuta as diferenças de acesso à memória e desempenho em relação à distribuição por linhas. Analise o desempenho das versões da tarefa 17, e das duas dessa tarefa.

2. DISCUSSÃO

Na tarefa 17, o produto matriz-vetor $y = A \cdot x$ foi calculado distribuindo a matriz `A` por linhas. Dessa forma, cada processo recebia algumas linhas completas da matriz e calculava diretamente uma parte do vetor `y`. Esse caso é mais simples, pois em `C` a matriz é armazenada por linhas na memória. Assim, ao percorrer uma linha, os elementos estão em posições contínuas de memória, o que favorece o acesso ao cache e reduz o custo de comunicação.

Nesta tarefa, a lógica foi modificada para distribuir a matriz por colunas. Nesse caso, cada processo recebe um bloco de colunas da matriz A e também o segmento correspondente do vetor x. Porém, diferente da divisão por linhas, cada processo não calcula apenas algumas posições de y. Agora, cada processo calcula uma contribuição parcial para todos os elementos de y.

Por exemplo, se a matriz A possui N colunas e existem P processos, cada processo recebe N/P colunas. Como cada elemento de y depende de todas as colunas da linha correspondente, cada processo calcula apenas uma parte da soma. Depois, todos esses vetores parciais são somados no processo 0 usando `MPI_Reduce` com `MPI_SUM`.

Dessa maneira, a operação fica organizada assim:

- 1º) A matriz A é dividida por colunas;
- 2º) O vetor x também é dividido em segmentos;
- 3º) Cada processo calcula um vetor parcial `y_parcial`, com M posições;
- 4º) O processo 0 soma todos os vetores parciais usando `MPI_Reduce`;
- 5º) O vetor y final é formado no processo 0.

A maior dificuldade dessa tarefa é que, em C, as colunas da matriz não ficam contínuas na memória. A matriz é armazenada por linhas, então os elementos de uma mesma linha ficam próximos, mas os elementos de uma mesma coluna ficam separados por um salto de N posições. Por isso, foi necessário usar tipos derivados do MPI para representar colunas.

2.1. MPI_Type_vector

O `MPI_Type_vector` permite criar um tipo derivado formado por blocos de dados separados por um salto fixo. Isso combina com o acesso por colunas, pois os elementos de uma coluna estão separados por N posições na matriz.

Nesta tarefa, o tipo vector foi usado para representar um bloco de colunas. Por exemplo, se cada processo recebe 300 colunas, o tipo derivado representa M linhas, cada uma contendo 300 elementos consecutivos, mas pulando N posições para chegar na próxima linha.

Contudo, ao usar apenas `MPI_Type_vector`, existe um problema importante: o tamanho real do tipo, chamado de extent, não fica igual ao tamanho do bloco de colunas. Ele fica grande, pois considera o espaço entre a primeira e a última

linha da matriz. Dessa forma, se esse tipo for usado diretamente com `MPI_Scatter`, os blocos enviados para os processos não começam exatamente na próxima coluna, mas sim em posições muito distantes da memória.

Por isso, na versão usando apenas `MPI_Type_vector`, foi necessário criar um buffer auxiliar no processo 0, deixando os blocos de colunas separados na memória de acordo com o extent natural do tipo. Essa versão funciona, mas é menos eficiente, pois usa mais memória e exige uma reorganização extra dos dados antes do `MPI_Scatter`.

2.2. `MPI_Type_create_resized`

A segunda versão utiliza `MPI_Type_vector` junto com `MPI_Type_create_resized`. O objetivo do `MPI_Type_create_resized` é alterar o extent do tipo derivado. Assim, mesmo que o tipo represente colunas com saltos dentro da matriz, o MPI passa a considerar que o próximo bloco começa logo após o bloco de colunas anterior.

Com isso, o `MPI_Scatter` consegue distribuir os blocos de colunas diretamente da matriz original, sem precisar criar um buffer auxiliar com espaços vazios. Portanto, essa versão tende a ser mais correta do ponto de vista de organização da memória e também mais eficiente.

- `MPI_Type_vector`: representa corretamente o padrão de colunas, mas possui extent grande;
- `MPI_Type_create_resized`: ajusta o extent, permitindo que o `MPI_Scatter` envie blocos consecutivos de colunas corretamente.

2.3. Diferença Entre Distribuição Por Linhas E Por Colunas

Na distribuição por linhas, cada processo recebe linhas completas da matriz. Isso é eficiente porque as linhas estão armazenadas de forma contínua na memória em C. Além disso, cada processo calcula diretamente uma parte final do vetor `y`, e no final basta juntar esses pedaços com `MPI_Gather`.

Na distribuição por colunas, a situação é diferente. Como as colunas são não-contíguas na memória, o MPI precisa usar tipos derivados para identificar corretamente esses dados. Além disso, cada processo calcula uma contribuição parcial para todos os elementos de `y`, e não apenas uma parte de

y. Por isso, no final é necessário usar MPI_Reduce com MPI_SUM para somar todos os vetores parciais.

Dessa forma, a distribuição por colunas geralmente possui maior custo de comunicação e acesso à memória menos favorável. Mesmo que cada processo receba menos elementos do vetor x, ele precisa gerar um vetor parcial de tamanho M e participar de uma redução global.

2.4. Speedup E Eficiência

O Speedup é uma métrica usada para verificar se o aumento da quantidade de processos realmente melhorou o desempenho do programa.

$\text{Speedup} = \text{Tempo com 1 processo} / \text{Tempo com N processos}$

Assim:

- Speedup > 1: o paralelismo ajudou;
- Speedup = 1: o paralelismo não trouxe ganho;
- Speedup < 1: o paralelismo piorou o desempenho;
- Speedup = N: seria o speedup ideal, mas raramente acontece na prática.

A Eficiência mostra o quanto os processos foram bem aproveitados.

$\text{Eficiência} = \text{Speedup} / \text{Número de processos}$

Para apresentar em porcentagem:

$\text{Eficiência (\%)} = (\text{Speedup} / \text{Número de processos}) * 100$

Nesta tarefa, essas métricas devem ser calculadas separadamente para as duas versões da tarefa 18 e depois comparadas com a tarefa 17.

2.5. Etapas Para Resolução Da Tarefa

Etapas para a resolução da tarefa:

- 1º) Relembrar a implementação da tarefa 17, que distribuiu a matriz por linhas;
- 2º) Modificar a lógica para dividir a matriz por colunas;
- 3º) Criar uma versão usando apenas MPI_Type_vector;

- 4º) Criar uma segunda versão usando `MPI_Type_vector` com `MPI_Type_create_resized`;
- 5º) Dividir o vetor `x` em segmentos com `MPI_Scatter`;
- 6º) Fazer cada processo calcular um vetor parcial `y_parcial` com `M` posições;
- 7º) Somar os vetores parciais com `MPI_Reduce` e `MPI_SUM`;
- 8º) Medir o tempo de execução;
- 9º) Calcular speedup e eficiência;
- 10º) Comparar o desempenho da tarefa 17 com as duas versões da tarefa 18.

2.6. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.7. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

Com a implementação da tarefa, foi possível observar que a forma como a matriz é dividida entre os processos influencia diretamente o desempenho do programa MPI.

Na tarefa 17, a distribuição por linhas foi mais natural para a linguagem C, pois as linhas da matriz ficam armazenadas de maneira contínua na memória. Dessa forma, cada processo recebeu um bloco contínuo de dados, calculou uma parte do vetor `y` e depois o processo 0 reuniu os resultados com `MPI_Gather`.

Na tarefa 18, a distribuição por colunas exigiu o uso de tipos derivados, pois as colunas da matriz não são contínuas na memória. A versão usando apenas `MPI_Type_vector` funcionou, mas exigiu um buffer auxiliar por causa do extent grande do tipo derivado. Já a versão usando `MPI_Type_create_resized` foi mais adequada, pois corrigiu o extent e permitiu distribuir os blocos de colunas de forma mais organizada.

Portanto, é possível concluir que o `MPI_Type_create_resized` é importante quando se deseja usar tipos derivados em operações coletivas como `MPI_Scatter`, principalmente quando os dados são não-contíguos na memória.

Além disso, os resultados reforçam que paralelizar um programa não depende apenas de dividir o trabalho entre processos. Também é necessário considerar o padrão de acesso à memória, a quantidade de comunicação, o custo das operações coletivas e a forma como os dados estão organizados fisicamente na memória.

4. APÊNDICE

4.1. RESOLUÇÃO USANDO VERSÃO `MPI_Type_vector`

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank;
    int quantidade_processos;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &quantidade_processos);

    int M = 1200;
    int N = 1200;
    double tempo_referencia = 0.0;

    if (argc >= 3) {
        M = atoi(argv[1]);
        N = atoi(argv[2]);
    }

    if (argc >= 4) {
        tempo_referencia = atof(argv[3]);
    }

    if (M <= 0 || N <= 0) {
        if (rank == 0) {
            printf("Erro: M e N precisam ser maiores que zero.\n");
        }

        MPI_Finalize();
        return 1;
    }
}
```

```

    if (N % quantidade_processos != 0) {
        if (rank == 0) {
            printf("Erro: N precisa ser divisivel pela quantidade de
processos.\n");
            printf("N = %d, processos = %d\n", N, quantidade_processos);
        }

        MPI_Finalize();
        return 1;
    }

    int colunas_por_processo = N / quantidade_processos;

    double *matriz_A = NULL;
    double *matriz_envio = NULL;
    double *matriz_local = NULL;

    double *vetor_x = NULL;
    double *vetor_x_local = NULL;

    double *vetor_y = NULL;
    double *vetor_y_parcial = NULL;

    matriz_local = (double *) malloc((size_t) M * colunas_por_processo *
sizeof(double));
    vetor_x_local = (double *) malloc(colunas_por_processo *
sizeof(double));
    vetor_y_parcial = (double *) malloc(M * sizeof(double));

    if (rank == 0) {
        matriz_A = (double *) malloc((size_t) M * N * sizeof(double));
        vetor_x = (double *) malloc(N * sizeof(double));
        vetor_y = (double *) malloc(M * sizeof(double));
    }

    int erro_memoria = 0;

    if (matriz_local == NULL || vetor_x_local == NULL || vetor_y_parcial ==
NULL) {
        erro_memoria = 1;
    }

    if (rank == 0 && (matriz_A == NULL || vetor_x == NULL || vetor_y ==
NULL)) {
        erro_memoria = 1;
    }

```

```

    int erro_global = 0;

    MPI_Allreduce(&erro_memoria, &erro_global, 1, MPI_INT, MPI_MAX,
MPI_COMM_WORLD);

    if (erro_global == 1) {
        if (rank == 0) {
            printf("Erro: faltou memoria.\n");
        }

        free(matriz_local);
        free(vetor_x_local);
        free(vetor_y_parcial);

        if (rank == 0) {
            free(matriz_A);
            free(vetor_x);
            free(vetor_y);
        }

        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        int i;
        int j;

        for (i = 0; i < M; i++) {
            for (j = 0; j < N; j++) {
                matriz_A[(size_t) i * N + j] = (double) ((i + j) % 10 + 1);
            }
        }

        for (j = 0; j < N; j++) {
            vetor_x[j] = 1.0;
        }
    }

    /*
    tipo derivado com MPI_Type_vector.

    Esse tipo representa um bloco de colunas
    count = M -> quantidade de linhas
    blocklength = colunas_por_processo -> quantas colunas seguidas em
cada linha
    stride = N -> salto para chegar na proxima linha da matriz original

```



```

    O problema é que o extent natural desse tipo fica grande.
    nessa versao, o processo 0 cria um buffer auxiliar.
*/
MPI_Datatype tipo_colunas;

MPI_Type_vector(M,colunas_por_processo,N,MPI_DOUBLE,&tipo_colunas);

MPI_Type_commit(&tipo_colunas);

MPI_Aint limite_inferior;
MPI_Aint extent_bytes;

MPI_Type_get_extent(tipo_colunas, &limite_inferior, &extent_bytes);

size_t extent_elementos = (size_t) (extent_bytes / sizeof(double));

if (rank == 0) {
    matriz_envio = (double *) malloc(extent_elementos *
quantidade_processos * sizeof(double));

    if (matriz_envio == NULL) {
        printf("Erro: faltou memoria para o buffer auxiliar da versao
vector.\n");

        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}

MPI_Barrier(MPI_COMM_WORLD);

double inicio_total = MPI_Wtime();

/*
    usando apenas MPI_Type_vector, o extent fica grande
    organiza manualmente o buffer de envio para combinar com esse
extent

    Isso nao é o mais eficiente, mas serve para mostrar a diferenca
    entre usar apenas MPI_Type_vector e usar MPI_Type_create_resized
*/
if (rank == 0) {
    int p;
    int i;
    int j;

    for (p = 0; p < quantidade_processos; p++) {
        size_t base = (size_t) p * extent_elementos;
        int coluna_inicial = p * colunas_por_processo;

```

```

        for (i = 0; i < M; i++) {
            for (j = 0; j < colunas_por_processo; j++) {
                matriz_envio[base + (size_t) i * N + j] =
                    matriz_A[(size_t) i * N + coluna_inicial + j];
            }
        }
    }

    MPI_Scatter(matriz_envio, 1, tipo_colunas, matriz_local, M *
colunas_por_processo, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    MPI_Scatter(vetor_x, colunas_por_processo, MPI_DOUBLE, vetor_x_local, colun
as_por_processo, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    double inicio_calculo = MPI_Wtime();

    /*
        Cada processo calcula uma contribuicao parcial para todos os
elementos de y.

        Como cada processo tem apenas algumas colunas, ele calcula:
y_parcial[i] = soma das colunas que esse processo recebeu
    */
    int i;
    int j;

    for (i = 0; i < M; i++) {
        double soma = 0.0;

        for (j = 0; j < colunas_por_processo; j++) {
            soma += matriz_local[(size_t) i * colunas_por_processo + j] *
vetor_x_local[j];
        }

        vetor_y_parcial[i] = soma;
    }

    double fim_calculo = MPI_Wtime();

    /*
        Agora todos os vetores parciais sao somados no processo 0
isso forma o vetor y final
    */
    MPI_Reduce(vetor_y_parcial, vetor_y, M, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORL
D);

```

```

double fim_total = MPI_Wtime();

double tempo_total_local = fim_total - inicio_total;
double tempo_calculo_local = fim_calculo - inicio_calculo;

double tempo_total = 0.0;
double tempo_calculo = 0.0;

MPI_Reduce(&tempo_total_local, &tempo_total, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);
MPI_Reduce(&tempo_calculo_local, &tempo_calculo, 1, MPI_DOUBLE,
MPI_MAX, 0, MPI_COMM_WORLD);

if (rank == 0) {
    double checksum = 0.0;

    for (i = 0; i < M; i++) {
        checksum += vetor_y[i];
    }

    printf("\nRESULTADO - MPI_TYPE_VECTOR\n");
    printf("Matriz A: %d x %d\n", M, N);
    printf("Processos: %d\n", quantidade_processos);
    printf("Colunas por processo: %d\n", colunas_por_processo);
    printf("Tempo total: %f segundos\n", tempo_total);
    printf("Tempo apenas do calculo local: %f segundos\n",
tempo_calculo);
    printf("Checksum do vetor y: %.2f\n", checksum);

    if (tempo_referencia > 0.0) {
        double speedup = tempo_referencia / tempo_total;
        double eficiencia = (speedup / quantidade_processos) * 100.0;

        printf("Tempo de referencia com 1 processo: %f segundos\n",
tempo_referencia);
        printf("Speedup: %.2fx\n", speedup);
        printf("Eficiencia: %.2f%%\n", eficiencia);
    } else {
        printf("Tempo de referencia nao informado.\n");
        printf("Se este teste foi com 1 processo, use o Tempo total
como referencia nos proximos testes.\n");
    }
}

MPI_Type_free(&tipo_colunas);

free(matriz_local);
free(vetor_x_local);

```

```

    free(vetor_y_parcial);

    if (rank == 0) {
        free(matriz_A);
        free(matriz_envio);
        free(vetor_x);
        free(vetor_y);
    }

    MPI_Finalize();

    return 0;
}

```

4.2. RESOLUÇÃO USANDO MPI_Type_vector + MPI_Type_create_resized

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank;
    int quantidade_processos;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &quantidade_processos);

    int M = 1200;
    int N = 1200;
    double tempo_referencia = 0.0;

    if (argc >= 3) {
        M = atoi(argv[1]);
        N = atoi(argv[2]);
    }

    if (argc >= 4) {
        tempo_referencia = atof(argv[3]);
    }

    if (M <= 0 || N <= 0) {
        if (rank == 0) {
            printf("Erro: M e N precisam ser maiores que zero.\n");

```

```

    }

    MPI_Finalize();
    return 1;
}

if (N % quantidade_processos != 0) {
    if (rank == 0) {
        printf("Erro: N precisa ser divisivel pela quantidade de
processos.\n");
        printf("N = %d, processos = %d\n", N, quantidade_processos);
    }

    MPI_Finalize();
    return 1;
}

int colunas_por_processo = N / quantidade_processos;

double *matriz_A = NULL;
double *matriz_local = NULL;

double *vetor_x = NULL;
double *vetor_x_local = NULL;

double *vetor_y = NULL;
double *vetor_y_parcial = NULL;

matriz_local = (double *) malloc((size_t) M * colunas_por_processo *
sizeof(double));
vetor_x_local = (double *) malloc(colunas_por_processo *
sizeof(double));
vetor_y_parcial = (double *) malloc(M * sizeof(double));

if (rank == 0) {
    matriz_A = (double *) malloc((size_t) M * N * sizeof(double));
    vetor_x = (double *) malloc(N * sizeof(double));
    vetor_y = (double *) malloc(M * sizeof(double));
}

int erro_memoria = 0;

if (matriz_local == NULL || vetor_x_local == NULL || vetor_y_parcial ==
NULL) {
    erro_memoria = 1;
}

```

```

    if (rank == 0 && (matriz_A == NULL || vetor_x == NULL || vetor_y ==
NULL)) {
        erro_memoria = 1;
    }

    int erro_global = 0;

    MPI_Allreduce(&erro_memoria, &erro_global, 1, MPI_INT, MPI_MAX,
MPI_COMM_WORLD);

    if (erro_global == 1) {
        if (rank == 0) {
            printf("Erro: faltou memoria.\n");
        }

        free(matriz_local);
        free(vetor_x_local);
        free(vetor_y_parcial);

        if (rank == 0) {
            free(matriz_A);
            free(vetor_x);
            free(vetor_y);
        }

        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        int i;
        int j;

        for (i = 0; i < M; i++) {
            for (j = 0; j < N; j++) {
                matriz_A[(size_t) i * N + j] = (double) ((i + j) % 10 + 1);
            }
        }

        for (j = 0; j < N; j++) {
            vetor_x[j] = 1.0;
        }
    }

    // cria o tipo vector normal-> representa um bloco de colunas, mas
    ainda tem o extent grande.

```

```

MPI_Datatype tipo_colunas_temp;
MPI_Datatype tipo_colunas_resized;

MPI_Type_vector(M,colunas_por_processo,
N,MPI_DOUBLE,&tipo_colunas_temp);

/*
ajusta o extent
quando o MPI_Scatter for enviar para o processo 0, 1, 2...
ele vai andar de colunas_por_processo em colunas_por_processo,
e nao pelo extent gigante do MPI_Type_vector original
*/
MPI_Type_create_resized(tipo_colunas_temp,0,colunas_por_processo *
sizeof(double),&tipo_colunas_resized);

MPI_Type_commit(&tipo_colunas_resized);

MPI_Barrier(MPI_COMM_WORLD);

double inicio_total = MPI_Wtime();

MPI_Scatter(matriz_A,1,tipo_colunas_resized,matriz_local,M *
colunas_por_processo,MPI_DOUBLE,0,MPI_COMM_WORLD);

MPI_Scatter(vetor_x,colunas_por_processo,MPI_DOUBLE,vetor_x_local,colun
as_por_processo,MPI_DOUBLE,0,MPI_COMM_WORLD);

double inicio_calculo = MPI_Wtime();

// Cada processo calcula uma contribuicao parcial para todos os
elementos de y.-> Depois essas contribuicoes vao ser somadas com MPI_Reduce

int i;
int j;

for (i = 0; i < M; i++) {
    double soma = 0.0;

    for (j = 0; j < colunas_por_processo; j++) {
        soma += matriz_local[(size_t) i * colunas_por_processo + j] *
vetor_x_local[j];
    }

    vetor_y_parcial[i] = soma;
}

double fim_calculo = MPI_Wtime();

```

```

    MPI_Reduce(vetor_y_parcial, vetor_y, M, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORL
D);

    double fim_total = MPI_Wtime();

    double tempo_total_local = fim_total - inicio_total;
    double tempo_calculo_local = fim_calculo - inicio_calculo;

    double tempo_total = 0.0;
    double tempo_calculo = 0.0;

    MPI_Reduce(&tempo_total_local, &tempo_total, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);
    MPI_Reduce(&tempo_calculo_local, &tempo_calculo, 1, MPI_DOUBLE,
MPI_MAX, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        double checksum = 0.0;

        for (i = 0; i < M; i++) {
            checksum += vetor_y[i];
        }

        printf("\nRESULTADO - MPI_TYPE_CREATE_RESIZED\n");
        printf("Matriz A: %d x %d\n", M, N);
        printf("Processos: %d\n", quantidade_processos);
        printf("Colunas por processo: %d\n", colunas_por_processo);
        printf("Tempo total: %f segundos\n", tempo_total);
        printf("Tempo apenas do calculo local: %f segundos\n",
tempo_calculo);
        printf("Checksum do vetor y: %.2f\n", checksum);

        if (tempo_referencia > 0.0) {
            double speedup = tempo_referencia / tempo_total;
            double eficiencia = (speedup / quantidade_processos) * 100.0;

            printf("Tempo de referencia com 1 processo: %f segundos\n",
tempo_referencia);
            printf("Speedup: %.2fx\n", speedup);
            printf("Eficiencia: %.2f%%\n", eficiencia);
        } else {
            printf("Tempo de referencia nao informado.\n");
        }
    }

    MPI_Type_free(&tipo_colunas_resized);
    MPI_Type_free(&tipo_colunas_temp);

```



```
    free(matriz_local);
    free(vetor_x_local);
    free(vetor_y_parcial);

    if (rank == 0) {
        free(matriz_A);
        free(vetor_x);
        free(vetor_y);
    }

    MPI_Finalize();

    return 0;
}
```

4.3. OUTPUT USANDO VERSÃO MPI_Type_vector

```

tarefa18A.obj
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 1 .\tarefa18A.exe 1200 1200

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 1200 x 1200
Processos: 1
Colunas por processo: 1200
Tempo total: 0.013635 segundos
Tempo apenas do calculo local: 0.000869 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia nao informado.
Se este teste foi com 1 processo, use o Tempo total como referencia nos proximos testes.
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 2 .\tarefa18A.exe 1200 1200 0.008837

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 1200 x 1200
Processos: 2
Colunas por processo: 600
Tempo total: 0.026611 segundos
Tempo apenas do calculo local: 0.001359 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.008837 segundos
Speedup: 0.33x
Eficiencia: 16.60%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 4 .\tarefa18A.exe 1200 1200 0.008837

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 1200 x 1200
Processos: 4
Colunas por processo: 300
Tempo total: 0.012898 segundos
Tempo apenas do calculo local: 0.000323 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.008837 segundos
Speedup: 0.69x
Eficiencia: 17.13%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 6 .\tarefa18A.exe 1200 1200 0.008837

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 1200 x 1200
Processos: 6
Colunas por processo: 200
Tempo total: 0.033276 segundos
Tempo apenas do calculo local: 0.000259 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.008837 segundos
Speedup: 0.27x
Eficiencia: 4.43%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 8 .\tarefa18A.exe 1200 1200 0.008837

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 1200 x 1200
Processos: 8
Colunas por processo: 150
Tempo total: 0.038810 segundos
Tempo apenas do calculo local: 0.000208 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.008837 segundos
Speedup: 0.23x
Eficiencia: 2.85%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 1 .\tarefa18A.exe 2400 2400

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 2400 x 2400
Processos: 1
Colunas por processo: 2400
Tempo total: 0.034325 segundos
Tempo apenas do calculo local: 0.005823 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia nao informado.
Se este teste foi com 1 processo, use o Tempo total como referencia nos proximos testes.
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 2 .\tarefa18A.exe 2400 2400 0.040122

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 2400 x 2400
Processos: 2
Colunas por processo: 1200
Tempo total: 0.079133 segundos

```

```
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.040122 segundos
Speedup: 0.51x
Eficiencia: 25.35%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 4 .\tarefa18A.exe 2400 2400 0.040122

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 2400 x 2400
Processos: 4
Colunas por processo: 600
Tempo total: 0.038037 segundos
Tempo apenas do calculo local: 0.001275 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.040122 segundos
Speedup: 1.05x
Eficiencia: 26.37%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 6 .\tarefa18A.exe 2400 2400 0.040122

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 2400 x 2400
Processos: 6
Colunas por processo: 400
Tempo total: 0.047775 segundos
Tempo apenas do calculo local: 0.001201 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.040122 segundos
Speedup: 0.84x
Eficiencia: 14.00%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 8 .\tarefa18A.exe 2400 2400 0.040122

RESULTADO - MPI_TYPE_VECTOR
Matriz A: 2400 x 2400
Processos: 8
Colunas por processo: 300
Tempo total: 0.086972 segundos
Tempo apenas do calculo local: 0.001852 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.040122 segundos
Speedup: 0.46x
Eficiencia: 5.77%
```

4.4. USANDO MPI_Type_vector + MPI_Type_create_resized

```

PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 1 .\tarefa18B.exe 1200 1200

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 1200 x 1200
Processos: 1
Colunas por processo: 1200
Tempo total: 0.007689 segundos
Tempo apenas do calculo local: 0.002353 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia nao informado.
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 2 .\tarefa18B.exe 1200 1200 0.006285

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 1200 x 1200
Processos: 2
Colunas por processo: 600
Tempo total: 0.004753 segundos
Tempo apenas do calculo local: 0.000424 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.006285 segundos
Speedup: 1.32x
Eficiencia: 66.11%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 4 .\tarefa18B.exe 1200 1200 0.006285

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 1200 x 1200
Processos: 4
Colunas por processo: 300
Tempo total: 0.005478 segundos
Tempo apenas do calculo local: 0.000326 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.006285 segundos
Speedup: 1.15x
Eficiencia: 28.68%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 6 .\tarefa18B.exe 1200 1200 0.006285

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 1200 x 1200
Processos: 6
Colunas por processo: 200
Tempo total: 0.005925 segundos

```

```

Tempo apenas do calculo local: 0.000221 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.006285 segundos
Speedup: 1.06x
Eficiencia: 17.68%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 8 .\tarefa18B.exe 1200 1200 0.006285

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 1200 x 1200
Processos: 8
Colunas por processo: 150
Tempo total: 0.014587 segundos
Tempo apenas do calculo local: 0.000334 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.006285 segundos
Speedup: 0.43x
Eficiencia: 5.39%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 1 .\tarefa18B.exe 2400 2400

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 2400 x 2400
Processos: 1
Colunas por processo: 2400
Tempo total: 0.048640 segundos
Tempo apenas do calculo local: 0.009323 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia nao informado.
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 2 .\tarefa18B.exe 2400 2400 0.015544

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 2400 x 2400
Processos: 2
Colunas por processo: 1200
Tempo total: 0.020490 segundos
Tempo apenas do calculo local: 0.002146 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.015544 segundos
Speedup: 0.76x
Eficiencia: 37.93%

```

```
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 2 .\tarefa18B.exe 2400 2400 0.015544
Tempo de referencia com 1 processo: 0.015544 segundos
Speedup: 0.76x
Eficiencia: 37.93%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 4 .\tarefa18B.exe 2400 2400 0.015544

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 2400 x 2400
Processos: 4
Colunas por processo: 600
Tempo total: 0.027971 segundos
Tempo apenas do calculo local: 0.001234 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.015544 segundos
Speedup: 0.56x
Eficiencia: 13.89%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 6 .\tarefa18B.exe 2400 2400 0.015544

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 2400 x 2400
Processos: 6
Colunas por processo: 400
Tempo total: 0.041397 segundos
Tempo apenas do calculo local: 0.001360 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.015544 segundos
Speedup: 0.38x
Eficiencia: 6.26%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> mpiexec -n 8 .\tarefa18B.exe 2400 2400 0.015544

RESULTADO - MPI_TYPE_CREATE_RESIZED
Matriz A: 2400 x 2400
Processos: 8
Colunas por processo: 300
Tempo total: 0.031583 segundos
Tempo apenas do calculo local: 0.001055 segundos
Checksum do vetor y: 31680000.00
Tempo de referencia com 1 processo: 0.015544 segundos
Speedup: 0.49x
Eficiencia: 6.15%
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa18> 
```